MINI PROJECT

Containerizing Sentiment Analysis using Docker

Presented By:-

Pradip R. Malik

Introduction

- In this project, we will containerize the sentiment analysis application using Docker, which can be deploy on Amazon Web Services (AWS) cloud for efficient and easy deployment. Sentiment analysis is a type of analysis that involves identifying and categorizing opinions expressed in text data. With the help of sentiment analysis, we can determine whether a piece of text is positive, negative, or neutral.
- Containerization involves packaging a software application and its dependencies into a container, which can then be run on any system. Containerization technology has revolutionized the software industry, and it has become popular due to its lightweight, scalable and efficient deployment capabilities.
- Additionally, cloud hosting permits multiple users to access the application simultaneously from different locations, providing a seamless and collaborative experience.

Problem Statement

- Without containerization, deploying sentiment analysis solutions can be challenging. Different operating systems can have different package dependencies, which can lead to compatibility issues. This can make deployment and scaling of sentiment analysis solutions more difficult and time-consuming. However, with Docker, these issues can be largely eliminated.

Objective

- containerize the sentiment analysis application using Docker and deploy it on AWS cloud.
- By containerizing the application, we can achieve better scalability, portability, and efficiency.
- The deployment on AWS cloud will help us in achieving easy deployment and quick scaling capabilities.

Components

Applications: Sentimental Analysis

Client: Web Browsers, Mobiles.

Platforms: AWS, Docker

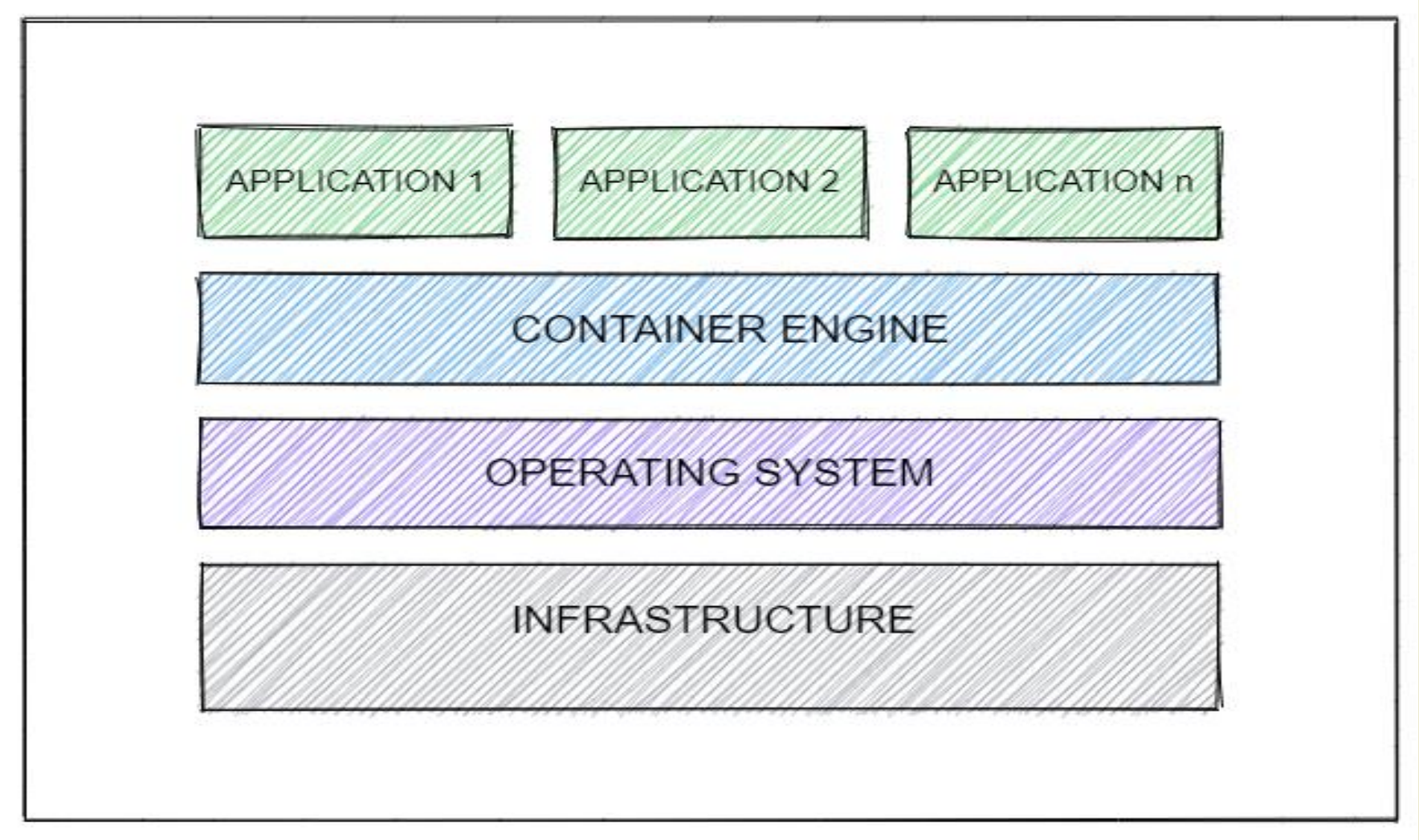
Services: Amazon Elastic Compute Cloud (EC2)

Storage: Postgress Database

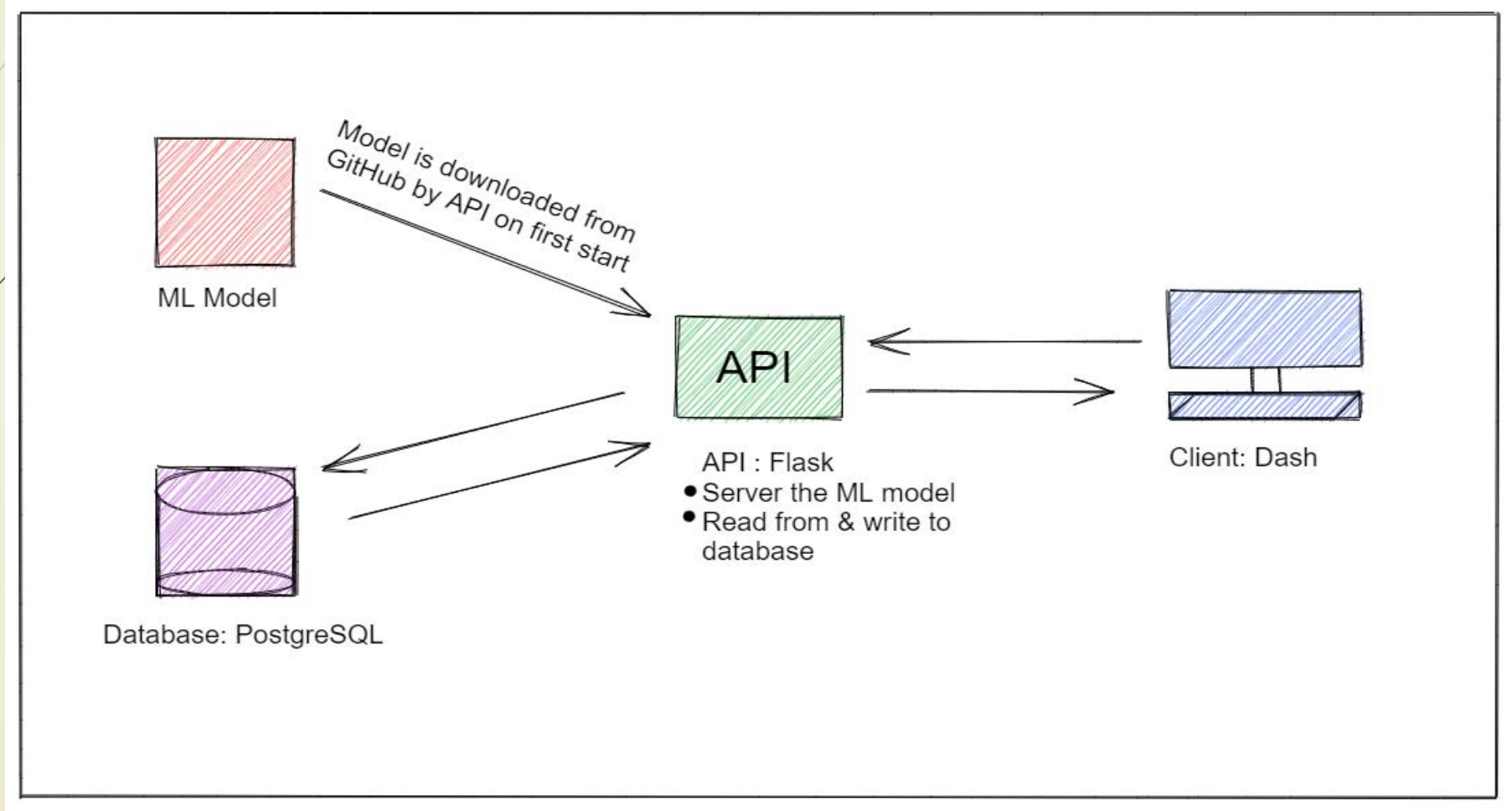
Advantages

- Reduce hardware coasts
- Portable
- Isolated
- deployment and scaling easier and faster
- Device independence
- Improve performance
- Reduce software coast
- Security

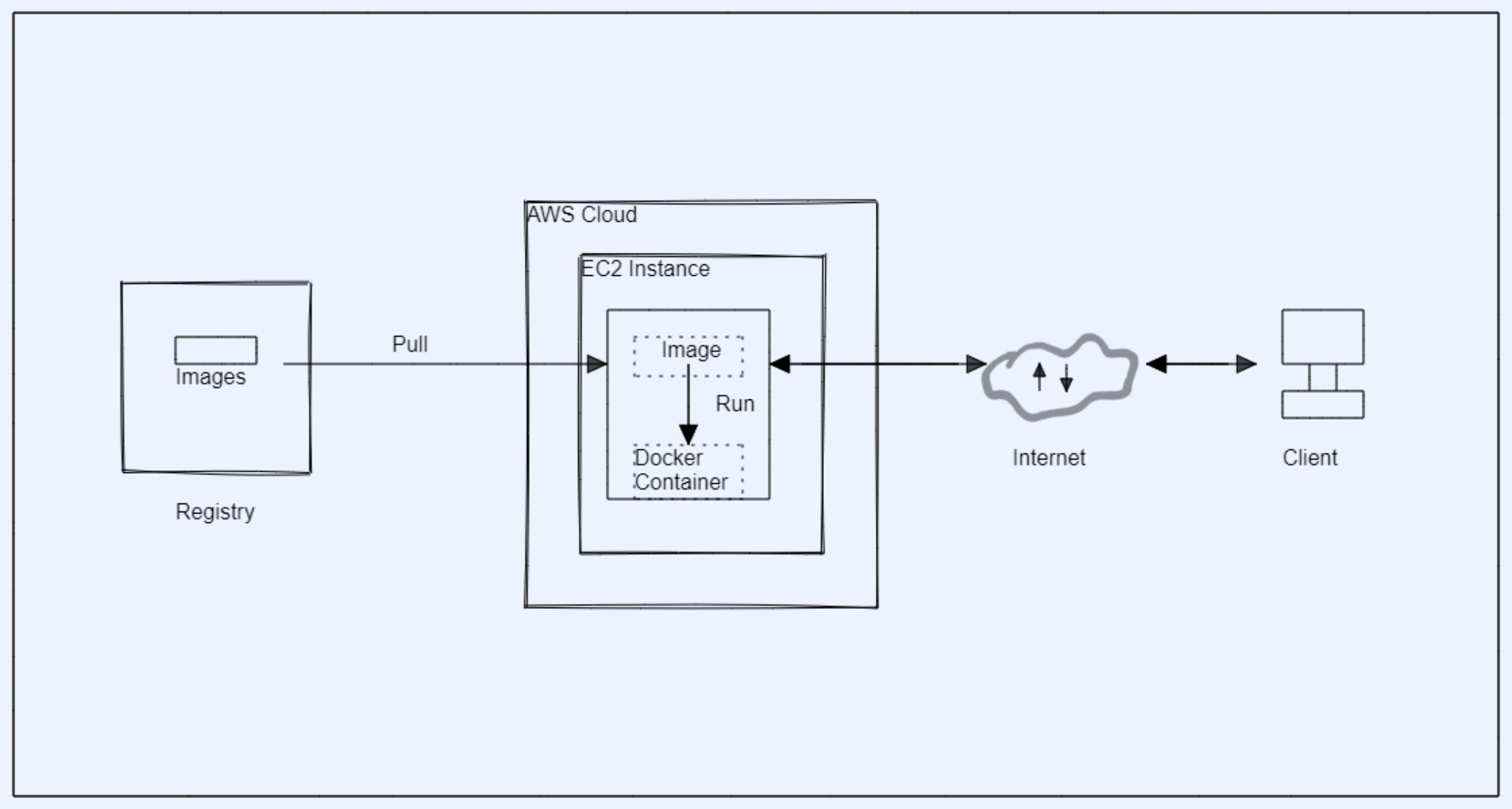
ARCHITECTURE OF CONTAINERIZATION



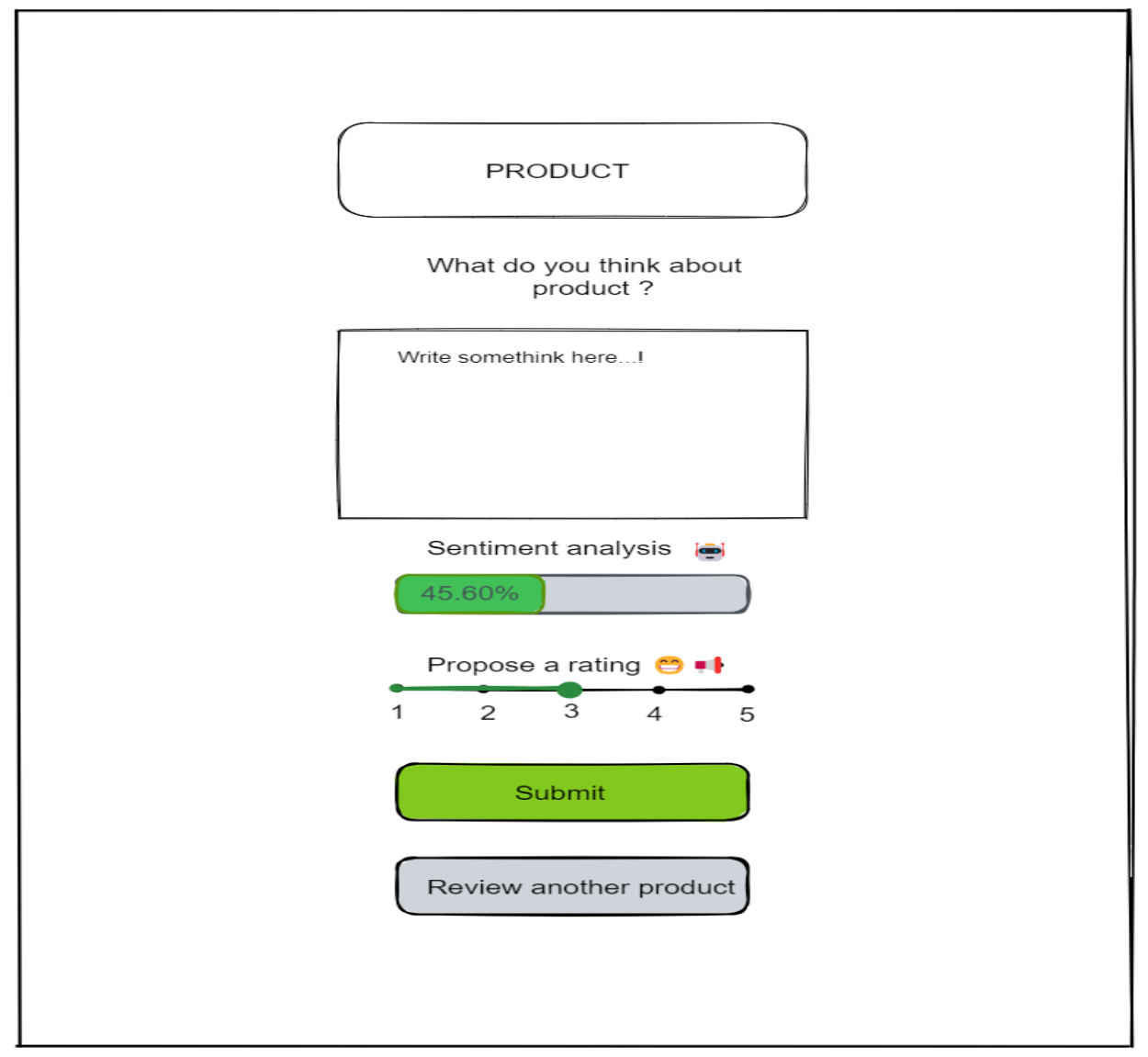
ML Application Schema



DEPLOYMENT WORKFLOW

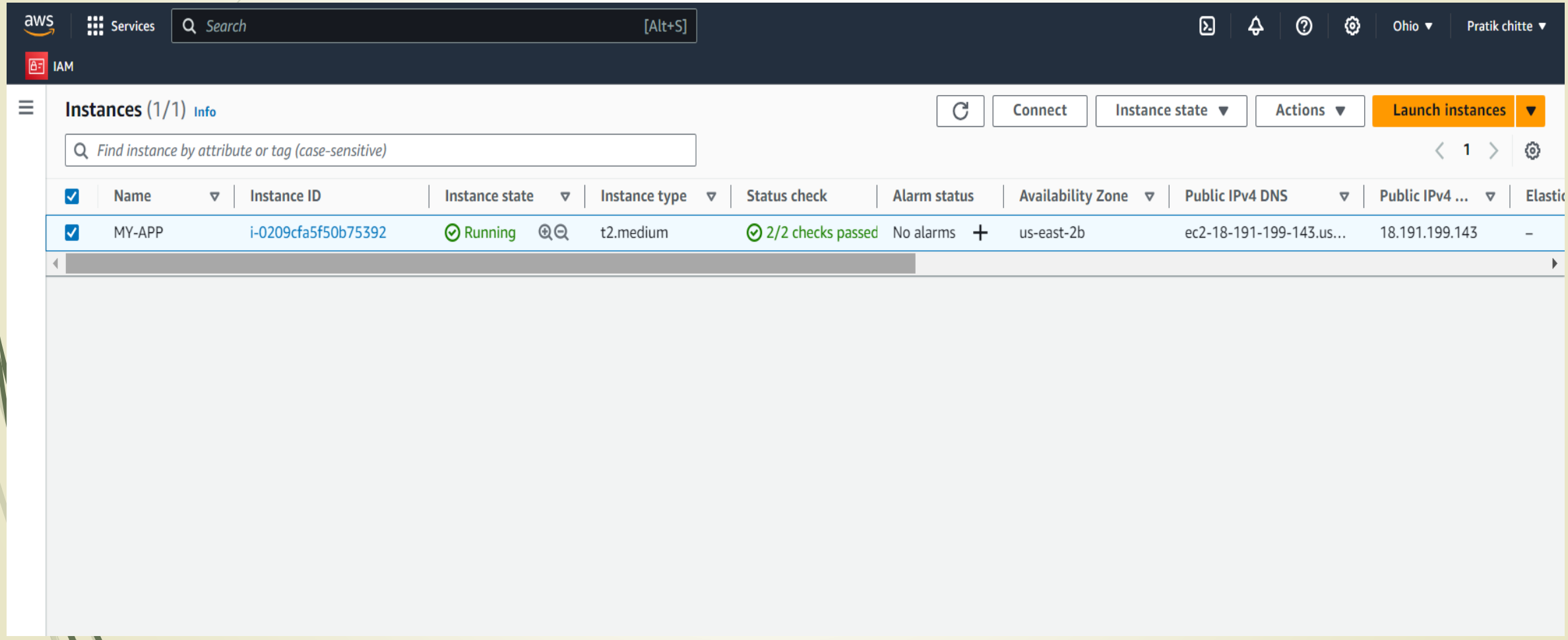


SENTIMENTAL ANALYSIS APPLICATION GUI

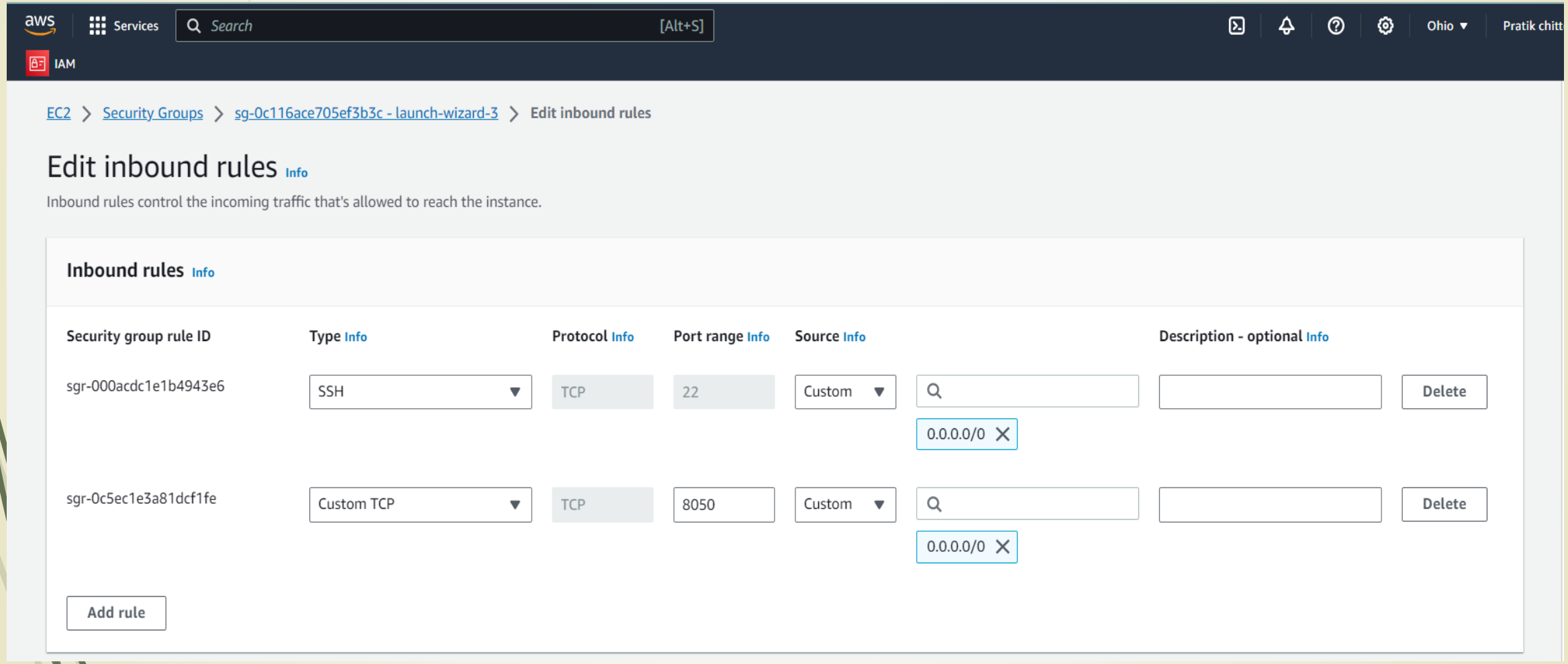


Working Model…

1.Create EC2 Ubuntu instance



2.Add TCP port numbers 8050 in security group



3.Install Docker on EC2 instance and run application using docker-compose

aws Services Search [Alt+S] Ohio Pratik chitte

IAM

```
ubuntu@ip-172-31-21-140:~/post-tuto-deployment$ sudo docker --version
Docker version 24.0.5, build 24.0.5-0ubuntu1~20.04.1
ubuntu@ip-172-31-21-140:~/post-tuto-deployment$
ubuntu@ip-172-31-21-140:~/post-tuto-deployment$ sudo docker images
REPOSITORY                  TAG       IMAGE ID       CREATED         SIZE
post-tuto-deployment_dash   latest    f79ec8146fc9   6 minutes ago   1.21GB
post-tuto-deployment_api    latest    55b281eea948   7 minutes ago   3.17GB
postgres                    latest    ec7f99c50d3c   6 days ago      418MB
python                      3.6       54260638d07c   21 months ago   902MB
ubuntu@ip-172-31-21-140:~/post-tuto-deployment$
ubuntu@ip-172-31-21-140:~/post-tuto-deployment$
ubuntu@ip-172-31-21-140:~/post-tuto-deployment$ sudo docker ps
CONTAINER ID   IMAGE                       COMMAND                  CREATED         STATUS          PORTS                                       NAMES
5f0c8ba621d6   post-tuto-deployment_dash   "gunicorn -b 0.0.0.0…"   4 minutes ago   Up 55 seconds   0.0.0.0:8050->8050/tcp, :::8050->8050/tcp   post-tuto-deployment_dash_1
d213c9a69cbc   post-tuto-deployment_api    "gunicorn -b 0.0.0.0…"   4 minutes ago   Up 55 seconds   5000/tcp                                    post-tuto-deployment_api_1
a3328d910d19   postgres                    "docker-entrypoint.s…"   4 minutes ago   Up 56 seconds   5432/tcp                                    post-tuto-deployment_db_1
ubuntu@ip-172-31-21-140:~/post-tuto-deployment$
```

i-0209cfa5f50b75392 (MY-APP)

PublicIPs: 18.191.199.143 PrivateIPs: 172.31.21.140

4.Postgres Back-end database shows the feedback of users

aws Services Search [Alt+S] Ohio Pratik chitte

IAM

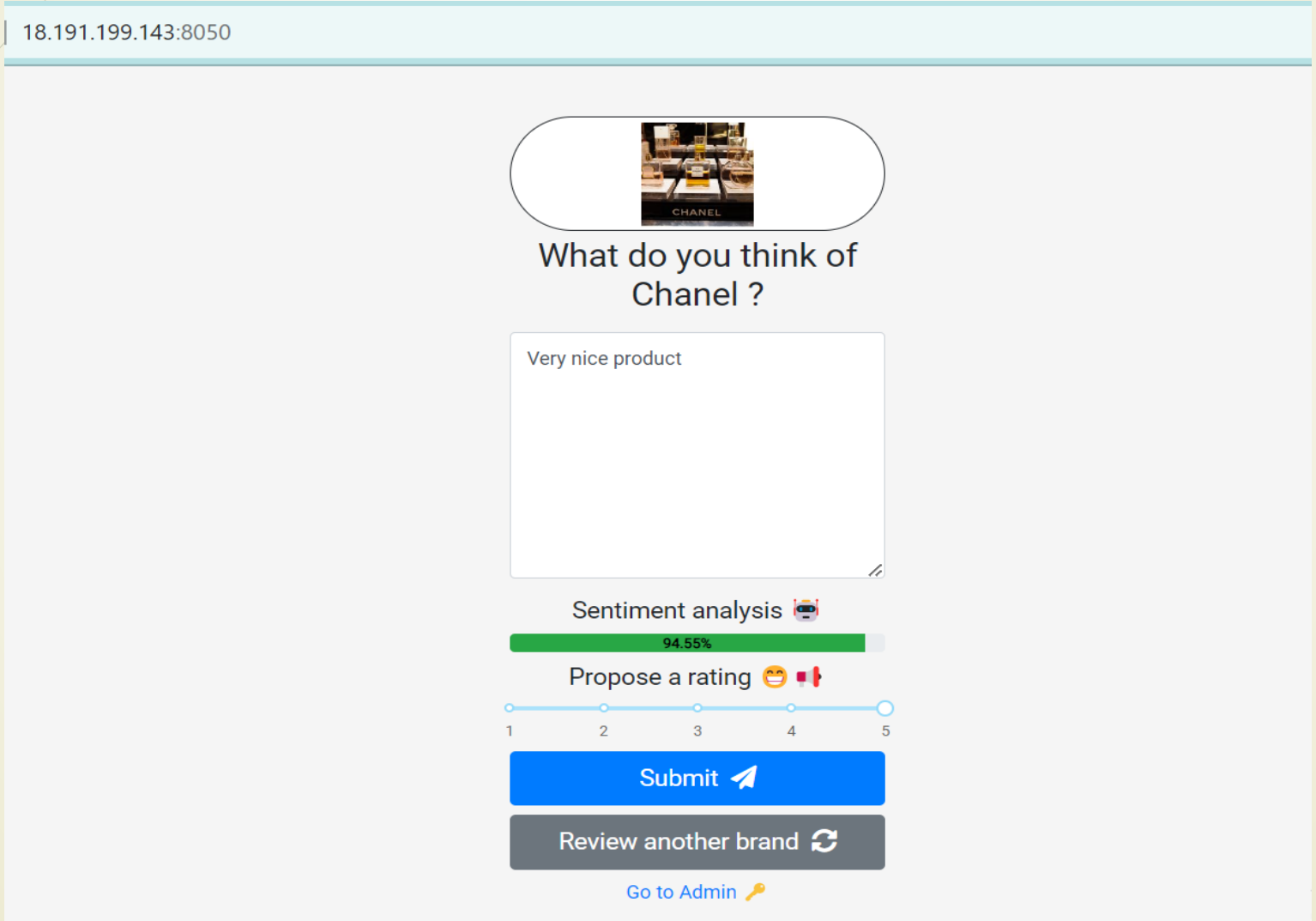
```
ubuntu@ip-172-31-21-140:~$ sudo docker exec -it  post-tuto-deployment_db_1 psql -U postgres
psql (16.0 (Debian 16.0-1.pgdg120+1))
Type "help" for help.

postgres=#  \dt
         List of relations
 Schema |  Name  | Type  |  Owner
--------+--------+-------+----------
 public | review | table | postgres
(1 row)

postgres=# \d review
                                          Table "public.review"
      Column      |            Type             | Collation | Nullable |              Default
------------------+-----------------------------+-----------+----------+-------------------------------------
 id               | integer                     |           | not null | nextval('review_id_seq'::regclass)
 review           | text                        |           | not null |
 rating           | integer                     |           | not null |
 suggested_rating | integer                     |           | not null |
 sentiment_score  | real                        |           | not null |
 brand            | text                        |           | not null |
 user_agent       | text                        |           | not null |
 ip_address       | text                        |           | not null |
 created_date     | timestamp without time zone |           | not null |
Indexes:
    "review_pkey" PRIMARY KEY, btree (id)

postgres=# SELECT * FROM review;
 id |      review       | rating | suggested_rating | sentiment_score |          brand            |                                                
```

5. Sentiment Analysis front-end application



6. Admin front-end application shows user feedback

Not secure | 18.191.199.143:8050/admin

Admin Page 🔑

id	brand	created_date	review	rating	suggested_rating	sentiment_score
3	Hewlett-Packard Enterprise	2023-09-26 17:40:07	Exellent	5	5	0.9533
2	Pepsi	2023-09-26 17:39:17	poor product	1	1	0.0547
1	Cartier	2023-09-26 17:36:52	Very nice product	5	5	0.9455

Go to Home 🏠

THANK YOU..